Tuple

In [ ]:
```
# Create a Tuple

thistuple = ("apple", "banana", "cherry")
print(thistuple)
```

In [ ]:
```
# Tuples allow duplicate values

thistuple = ("apple", "banana", "cherry", "apple", "cherry")
print(thistuple)
```

In [ ]:
```
# Tuple Length
# To determine how many items a tuple has, use the len() function

# Print the number of items in the tuple

thistuple = ("apple", "banana", "cherry")
print(len(thistuple))
```

In [ ]:
```
# Create Tuple With One Item
# To create a tuple with only one item, you have to add a comma after the item,otherwise Python will not recognize



thistuple = ("apple",)
print(type(thistuple))

#NOT a tuple
thistuple = ("apple")
print(type(thistuple))
```

In [ ]:
```
# Tuple Items - Data Types
# Tuple items can be of any data type:

tuple1 = ("apple", "banana", "cherry")
tuple2 = (1, 5, 7, 9, 3)
tuple3 = (True, False, False)
```

In [ ]:
```
# The data type of a tuple

mytuple = ("apple", "banana", "cherry")
print(type(mytuple))
```

In [ ]:
```
# The tuple() Constructor
# It is possible to use the tuple() constructor to make a tuple.


# Use the tuple() method to make a tuple:

thistuple = tuple(("apple", "banana", "cherry")) # note the double round-brackets
print(thistuple)
```

Access Tuple

In [ ]:
```
# Print the second item in the tuple:

thistuple = ("apple", "banana", "cherry")
print(thistuple[1])
```

In [ ]:
```
# Negative Indexing

# Print the last item of the tuple:

thistuple = ("apple", "banana", "cherry")
print(thistuple[-1])
```

In [ ]:
```python
# Range of Indexes
# Return the third, fourth, and fifth item:

thistuple = ("apple", "banana", "cherry", "orange", "kiwi", "melon", "mango")
print(thistuple[2:5])
```

In [ ]:
```python


thistuple = ("apple", "banana", "cherry", "orange", "kiwi", "melon", "mango")
print(thistuple[:4])
```

In [ ]:
```python
thistuple = ("apple", "banana", "cherry", "orange", "kiwi", "melon", "mango")
print(thistuple[2:])
```

In [ ]:
```python
 # Range of Negative Indexes
thistuple = ("apple", "banana", "cherry", "orange", "kiwi", "melon", "mango")
print(thistuple[-4:-1])
```

In [ ]:
```python
# Check if Item Exists

# Check if "apple" is present in the tuple:

thistuple = ("apple", "banana", "cherry")
if "apple" in thistuple:
  print("Yes, 'apple' is in the fruits tuple")
```

Update Tuple

In [ ]:
```python
# Change Tuple Values

# Once a tuple is created, you cannot change its values. Tuples are unchangeable, or immutable as it also is called

# Convert the tuple into a list to be able to change it

x = ("apple", "banana", "cherry")
y = list(x)
y[1] = "kiwi"
x = tuple(y)

print(x)
```

In [ ]:
```python
# Add Items
# Convert the tuple into a list, add "orange", and convert it back into a tuple:

thistuple = ("apple", "banana", "cherry")
y = list(thistuple)
y.append("orange")
thistuple = tuple(y)
print(thistuple)
```

In [ ]:
```python
# Add tuple to a tuple
# Create a new tuple with the value "orange", and add that tuple:

thistuple = ("apple", "banana", "cherry")
y = ("orange",)
thistuple += y

print(thistuple)
```

In [ ]:
```python
#Remove Items

# Tuples are unchangeable, so you cannot remove items from it, but you can use the same workaround as we used for c

#Convert the tuple into a list, remove "apple", and convert it back into a tuple:

thistuple = ("apple", "banana", "cherry")
y = list(thistuple)
y.remove("apple")
thistuple = tuple(y)

print(thistuple)
```

In [ ]:
```
#The del keyword can delete the tuple completely:

thistuple = ("apple", "banana", "cherry")
del thistuple
print(thistuple)
```

Unpack Tuples

In [ ]:
```
# Packing a tuple:

fruits = ("apple", "banana", "cherry")
print(fruits)
```

In [ ]:
```
# Unpacking a tuple:

fruits = ("apple", "banana", "cherry")

(green, yellow, red) = fruits

print(green)
print(yellow)
print(red)
```

In [ ]:
```
# Using Asterisk*

#If the number of variables is less than the number of values, you can add an * to the variable name and the values


# Assign the rest of the values as a list called "red"

fruits = ("apple", "banana", "cherry", "strawberry", "raspberry")

(green, yellow, *red) = fruits

print(green)
print(yellow)
print(red)
```

In [ ]:
```
# If the asterisk is added to another variable name than the last, Python will assign values to the variable until


# Add a list of values the "tropic" variable

fruits = ("apple", "mango", "papaya", "pineapple", "cherry")

(green, *tropic, red) = fruits

print(green)
print(tropic)
print(red)
```

Loop Tuples

In [ ]:
```
# Loop Through a Tuple

# You can loop through the tuple items by using a for loop.


# Iterate through the items and print the values

thistuple = ("apple", "banana", "cherry")
for x in thistuple:
 print(x)
```

In [ ]:
```
# Loop Through the Index Numbers

# You can also loop through the tuple items by referring to their index number.Use the range() and len() functions


# Print all items by referring to their index number

thistuple = ("apple", "banana", "cherry")
for i in range(len(thistuple)):
  print(thistuple[i])
```

In [ ]:
```
# Using a While Loop

#You can loop through the tuple items by using a while loop.


 # Print all items, using a while loop to go through all the index numbers:

thistuple = ("apple", "banana", "cherry")
i = 0
while i < len(thistuple):
  print(thistuple[i])
  i = i + 1
```

Join Tuples

In [ ]:
```
# Join Two Tuples
# To join two or more tuples you can use the + operator:


# Join two tuples

tuple1 = ("a", "b" , "c")
tuple2 = (1, 2, 3)

tuple3 = tuple1 + tuple2
print(tuple3)
```

In [ ]:
```
# Multiply Tuples

# If you want to multiply the content of a tuple a given number of times, you can use the * operator:


 # Multiply the fruits tuple by 2:

fruits = ("apple", "banana", "cherry")
mytuple = fruits * 2

print(mytuple)
```

Tuple Methods

Python has two built-in methods that you can use on tuples.

count() - Returns the number of times a specified value occurs in a tuple

index() - Searches the tuple for a specified value and returns the position of where it was found

In [ ]: